

PROJECT SCOPE STATEMENT	
Project Name	Ecosystem Discovery Application
Project Deliverables	Detailed Description
Freshwater Ecosystem Interface	A web-based interactive ecosystem scene representing a single Saskatchewan-inspired freshwater environment. The interface displays animated animals positioned within ecological zones such as underwater, shoreline, and surface areas. Users interact with the system through a structured gameplay interface rather than free exploration. The system includes a hint-based game where users select mystery cards and identify animals using clues within the ecosystem scene. The design emphasizes child-friendly usability, including: • Large clickable areas • Bright, readable visuals • Simple and intuitive navigation suitable for children ages 7–9
Animal Information Cards	Animal Discovery System When a user selects a mystery card, the system provides progressively revealed hints related to a hidden animal. Users must identify the correct animal in the ecosystem scene based on these hints. The system provides: • Multiple hints displayed step-by-step • Visual and interactive feedback for correct and incorrect selections • Educational information embedded within hints rather than separate pop-up cards This interaction replaces traditional pop-up information cards and supports guided learning through gameplay.
Sticker-Book Collection Page	The application includes a digital sticker-book panel that tracks gameplay progress. Features include: • Correctly identified animals appear as unlocked stickers • Failed or unsolved animals remain hidden or locked • A progress indicator showing number of animals completed • Visual feedback through animations and card flipping

	This feature reinforces learning through reward-based interaction within the game.
Structured Data Model	The system uses a structured backend data model storing: • Animal information (name, slug, image reference, active status) • Hint data linked to each animal • Game state information managed during runtime Animal and hint data are retrieved dynamically from a backend database (Supabase) using REST API calls. Runtime game state (e.g., active card, revealed hints, solved/failed status) is managed within the application using a centralized state module.
MVC Architecture implementation	The application follows the Model–View–Controller (MVC) architectural pattern: Model • Stores static page data and dynamic animal/hint data • Maintains runtime game state View • Renders the ecosystem scene, sticker book, and UI components • Displays animations, hints, and feedback Controller • Handles user interactions such as card selection, animal identification, and game flow • Coordinates data flow between Model and View The system is implemented using modular JavaScript files organized into models, views, controllers, services, and state management. This architecture ensures the system is organized, maintainable, and scalable.
Functional MVP Application	The project delivers a working Minimum Viable Product (MVP) demonstrating a complete interactive gameplay cycle: Start Game → Select Card → View Hints → Identify Animal → Receive Feedback → Unlock Sticker → Update Progress → Repeat until completion or timer expiry The MVP includes: • A hint-based animal discovery game • A countdown timer system • A sticker-book reward system

	• A bonus interaction (e.g., trash collection for extra time) • A videos page for additional learning This approach enhances engagement while maintaining educational value.
Project Documentation	The project deliverables will include formal documentation: • Stakeholder Register • Project Requirements Document • Updated Scope Statement • UML-style diagrams (process, data flow, class) • MVP plan and task breakdown
Version Control Repository	The project will maintain a structured GitHub repository containing: • All source code • Organized documentation folders • Project README • GitHub Project Board (Kanban) for task tracking
Project Exclusions	
The following items are explicitly outside the scope of this project: • Multiplayer functionality • User accounts or authentication systems • Advanced physics simulations or 3D game engines • Multiple ecosystems or habitat environments • More than 6 animals in the ecosystem • Complex scoring or competitive game mechanics • Long-term data persistence across sessions	